Configure agent hooks in Visual Studio Code (Preview)

Hooks run custom actions at specific points in an agent's lifecycle. Use them to add context, validate a tool call, run a formatter, or record an event without relying on the model to request the action.

This article helps you configure hooks for the shared Copilot runtime, choose a personal or shared source, and verify the result. If you already have hooks, start with Migrate hooks. Migration includes user folders, settings, custom agents, and plugins, not only repository files.

> **Note**
>
> The VS Code hooks experience is in Preview. Individual provider implementations can have a different lifecycle status and release schedule.

> **Important**
>
> Your organization might restrict which hooks can run. Claude and Codex harnesses are disabled when Copilot enterprise-managed settings apply. Contact the IT team that deployed the settings to discuss access. Administrators can learn how to manage hooks in enterprise environments.

Choose the hook implementation for your session

Select **Copilot** from the **Session Target** UI control to use the Copilot harness. VS Code, the GitHub Copilot app, and GitHub Copilot CLI share this harness and its hook format. You do not need to install or use the CLI to configure hooks in the editor.

Use the Copilot hooks reference for command configuration, event inputs, and outputs.

Other session targets use their own hook implementations:

Expand table

Session Target choice	Configuration and event reference
Claude	Claude hooks reference
Codex	Codex hooks documentation

Cloud sessions use the Copilot harness in a managed environment. Local hook files aren't automatically copied to that environment. Check the configuration available in the cloud workspace rather than assuming that hooks on your machine apply there.

The older Local harness has a separate Local hooks reference. Use it to interpret existing hooks during migration, not as the schema for a new Copilot hook.

Migrate hooks between harnesses

Sharing a hook file location does not establish runtime compatibility. Event naming can select a different payload, tool names and arguments can change, and output or error behavior can differ.

Follow the hook migration procedure and compatibility matrix before reusing a hook. Select the destination harness and start a test session; changing only the language model does not test migration.

Create and manage hooks

Select the harness before opening the hooks interface so the editor shows the relevant customization sources.

1. Open the Chat view or Agents window.
2. Select **Copilot** from the **Session Target** UI control.
3. Open the Agent Customizations editor:
   - In the Chat view, select **Open Customizations** in the title bar, and then **Hooks**.
   - In the Agents window, select **Customizations** in the sidebar, and then **Hooks** in the editor.
   - In the Chat view, enter `/hooks` in the chat input.
4. Review the sources and select a hook to open its configuration.

To generate a hook with AI, describe the hook you want in chat. The `/create-hook <description>` and **Chat: Generate Hook** shortcuts are available only for Local sessions. Review generated configuration and

scripts against the destination reference before running them. Generation is not automatic migration or verification.

Choose a hook source

Choose the scope that matches the workflow:

Expand table

Need	Source
Personal automation across repositories	User hook files in `~/.copilot/hooks/`, or `$COPILOT_HOME/hooks/` when `COPILOT_HOME` is set.
Automation shared with a repository	`.github/hooks/*.json` at the repository root
Hooks alongside other personal or repository settings	An inline `hooks` map in a supported Copilot settings file
A reusable setup with scripts and other customizations	Agent plugins
Automation active only for one custom agent	Agent-scoped hooks
Organization-managed validation	Enterprise hook controls

The Copilot harness reads inline hook settings in `.github/copilot/settings.json`, `.github/copilot/settings.local.json`, and `~/.copilot/settings.json`. It also reads repository `.claude/settings.json` and `.claude/settings.local.json`. User `config.json` is not the hook settings file. For all supported sources, precedence, and policy directories, see Copilot hook locations.

Applicable hooks from multiple sources run together. A new repository hook does not replace a user or plugin hook for the same event. Check for duplicates before enabling a migrated copy.

Also preserve disabled state. Local settings such as `chat.useHooks` do not prevent the Copilot harness from loading a configuration. Follow the preparation steps before switching.

In remote sessions and containers, the user folder and executable dependencies belong to the environment where the runtime runs. Local editor settings such as `chat.hookFilesLocations` do not configure Copilot discovery.

Create your first Copilot hook

Start with a diagnostic that records only the tool name, not its arguments or result. This example requires Node.js on the execution host.

Create `.github/hooks/audit.json`:

```
{
  "version": 1,
  "hooks": {
    "preToolUse": [
      {
        "type": "command",
        "command": "node .github/hooks/log-tool-use.cjs",
        "timeoutSec": 10
      }
    ]
  }
}
```

Create `.github/hooks/log-tool-use.cjs`:

```
const fs = require('node:fs');
const path = require('node:path');
const event = JSON.parse(fs.readFileSync(0, 'utf8'));
if (typeof event.toolName !== 'string') {
  throw new Error('Expected a Copilot preToolUse event with toolName.');
}
fs.appendFileSync(
```

```
  path.join(__dirname, 'tool-use.log'),
  `${JSON.stringify(event.toolName)}\n`
);
process.stdout.write('{}');
```

Add `.github/hooks/tool-use.log` to `.gitignore`. Start a new **Copilot** session and request a harmless task that uses a tool, such as reading a project file. Confirm that the log contains the tool name and that the task continues through its normal permission checks. Returning `{}` makes no permission decision; it does not automatically approve the tool.

The log is written beside the script, so moving the script to your user hook directory keeps its log there too. See the personal hook and settings examples for quoted command paths. For shared plugin hooks, use the bundled-script example; do not write logs into an installed plugin's code directory.

Configure commands and event data

A command hook receives event data as JSON on standard input (stdin). It can return one JSON output object on standard output (stdout). Keep diagnostic text separate from that output.

The Copilot harness supports a cross-platform `command` fallback and platform-specific `bash` or `powershell` commands. Command arguments belong in the command string. A shell-free `exec` and `args` form is also available; these arguments are not a replacement for the event JSON on stdin.

Use `cwd`, `env`, and `timeoutSec` only as documented in the command-hook reference. Review the command's dependencies and working directory when moving it to a user folder, plugin, remote host, or container.

Events and payloads

Choose an event for the action you need:

Expand table

Goal	Copilot event
Add context when a session starts or resumes	`sessionStart`
Observe a submitted prompt	`userPromptSubmitted`
Inspect or make a decision about a tool call	`preToolUse`
Validate a successful result or add context	`postToolUse`
Save information before compaction	`preCompact`
Observe a subagent starting or finishing	`subagentStart`, `subagentStop`
Check whether the agent should finish its turn	`agentStop`
Clean up when the session terminates	`sessionEnd`

See the event reference for the full event list, timing, fields, and subagent coverage. Not all outputs are consumed by every event. For example, command-hook output is ignored for `userPromptSubmitted`; programmatic SDK prompt rewriting is a different API.

Event names also select the input format. `preToolUse` receives `toolName` and `toolArgs`. The compatible `PreToolUse` alias receives snake_case fields, but can report Claude-style tool names such as `Read`, `Write`, and `Edit`. Neither format guarantees the same tool arguments as an old Local hook.

Decisions and failures

For `preToolUse`, return a documented top-level decision:

```
{
  "permissionDecision": "deny",
  "permissionDecisionReason": "This operation requires manual review."
}
```

Return `{}` for no decision. `ask` requires an interactive approval flow; do not assume it will pause an unattended session for approval. A `postToolUse` hook can add top-level `additionalContext`, but cannot undo an action that has already completed.

Treat error handling as part of the hook contract. GitHub documents command `preToolUse` errors as denying the tool call, but timeouts as falling through to normal permissions, including for policy hooks. Other events and HTTP hooks have different failure behavior. Test the exit-code and timeout rules instead of

assuming that every failure blocks work.

Agent-scoped hooks

An agent-scoped hook is declared in a custom agent's `hooks` frontmatter and applies while that agent is active. It runs in addition to applicable hooks from other sources.

The Copilot agent-scoped hooks reference describes supported command fields, trust requirements, subagent behavior, and exclusions for remote or organization-hosted definitions.

VS Code preserves the `hooks` field when converting custom agents for Copilot, including plugin-provided agents. Hook mappings remain scoped to their individual agents.

The custom-agent editor validates hook structure, but selecting an agent or seeing no validation hints doesn't prove that its hooks execute. Verify the expected event and any tool-denial behavior in a trusted session before relying on a hook.

Keep agent-specific automation in the agent's frontmatter. Moving a command to a repository-wide or plugin-wide event changes which agents trigger it and is not an equivalent configuration.

Existing Local hooks

Use this section only to identify an existing Local setup before migrating it.

- Local discovers repository and user hook files, configured `chat.hookFilesLocations`, and agent or plugin hooks. Claude settings files require `chat.useClaudeHooks`.
- `chat.useHooks` controls Local execution. These settings do not configure the shared Copilot runtime.
- The Local parser accepts PascalCase and Copilot event aliases, command aliases, and nested Claude command groups. It ignores matcher values and still sends Local payloads.
- Local agent-scoped hooks require a trusted workspace and `chat.useHooks`. A scoped `Stop` hook is treated as `SubagentStop` when the agent runs as a subagent.

The Local reference retains the configuration, input, and output details needed to convert those scripts. For new hooks, use the Copilot examples above.

Troubleshoot hooks

1. Confirm the **Session Target**, installed runtime version, and hook source.
2. Check whether another user, repository, settings, agent, or plugin source contributes the same command.
3. Run the script directly with a non-sensitive event fixture. Check its exit status and exact JSON output, including malformed-input behavior.
4. Start a disposable session and confirm the hook fires for the expected event and tool. A file shown in the editor is not proof that its script executed.

5. Check dependencies, path quoting, `cwd`, and permissions in the execution environment. Test timeouts separately from command errors.

Use the agent debug logs and the provider's diagnostics to investigate. Avoid logging complete prompts, arguments, results, or environment variables that might contain sensitive data.

For an old Local setup, **Chat: Configure Hooks** lists its files, and the **GitHub Copilot Chat Hooks** output channel shows its command errors.

Security considerations

> **Caution**
>
> Hook configuration and scripts are executable code. Review them before use, including hooks bundled in plugins or stored in a shared repository.

- Validate event input before using it. Do not interpolate untrusted tool arguments into a shell command.
- Give scripts only the filesystem and network access they need. A hook's `cwd` or `env` does not widen sandbox access.
- Keep credentials out of configuration, scripts, output, logs, and agent context.
- Protect the configuration and scripts against unintended changes.

- Verify source restrictions, failure behavior, and event coverage in each destination. Hooks and instructions are not substitutes for enforced permissions, sandboxing, or enterprise controls.

Related resources

- Migrate hooks to the Copilot harness
- GitHub Copilot hooks reference
- Security considerations for AI agents

Was this page helpful?

Yes No

Still need help?

Ask the community

Request features

Report issues

Help us improve

Edit this page

10/28/2026